

LLP TCA2 — Complete Detailed Study Notes

1. GCC Compilation & Toolchain (AA1.3)

What is GCC?

GCC (GNU Compiler Collection) is a command-line compiler for C programs. It translates your `.c` source code into an executable through multiple stages.

Compilation Stages

Stage 1 — Pre-processing (`.i` file)

- Expands `#include` headers and `#define` macros
- Removes comments
- Command:

```
gcc -E Greeting.c -o Greeting.i
```

Stage 2 — Compilation to Assembly (`.s` file)

- Converts C code into human-readable assembly language
- Command:

```
gcc -S Greeting.c -o Greeting.s
```

Stage 3 — Compile to Executable (`.exe` file)

- Full compilation: pre-process → compile → assemble → link
- Command:

```
gcc Greeting.c -o Greeting.exe
```

Run the executable:

```
Greeting.exe
```

Compiling multiple files into one executable:

```
gcc main.c myheader.c -o myapp.exe
```

This is why header files are important — they allow code to be split across multiple `.c` files while sharing function declarations.

Why Header Files Are Important

- Separate declaration (what a function is) from definition (what it does)
- Allow multiple `.c` files to share the same function signatures and struct definitions
- Prevent code duplication
- Enable modular, maintainable programs

2. Header Files (AA1.3 / SE4.3)

Creating a Header File

A header file (`.h`) contains:

- Struct definitions
- Function prototypes (declarations only — no body)
- `#define` constants
- Include guards to prevent double-inclusion

Include Guards — CRITICAL

```
#ifndef MYHEADER_H      // "if not defined"
#define MYHEADER_H      // define it now

// your struct declarations and function prototypes here

#endif                 // end of guard
```

Without include guards, if two files both include the same header, the compiler sees duplicate definitions and throws an error.

Full Header File Example

```
// myHeader.h
#ifndef MYHEADER_H
#define MYHEADER_H

#include <stdio.h>

// Struct declaration
struct dog {
    char chipId[15];
    char name[30];
    char breed[30];
};
```

```
};

// Function prototypes
struct dog fillRecord();
void displayRecord(struct dog d);

#endif
```

Corresponding .c file (myHeader.c or functions.c)

```
// myHeader.c
#include "myHeader.h"    // use quotes for your own files
#include <stdio.h>

struct dog fillRecord() {
    struct dog d;
    printf("Enter chip ID: ");
    scanf("%14s", d.chipId);
    printf("Enter name: ");
    scanf("%29s", d.name);
    printf("Enter breed: ");
    scanf("%29s", d.breed);
    return d;
}

void displayRecord(struct dog d) {
    printf("Chip ID: %s\n", d.chipId);
    printf("Name: %s\n", d.name);
    printf("Breed: %s\n", d.breed);
}
```

Using the header in main.c

```
// main.c
#include "myHeader.h"    // quotes = your own file
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    struct dog d = fillRecord();
    displayRecord(d);
    return 0;
}
```

Compile command:

```
gcc main.c myHeader.c -o app.exe
```

3. Visual Studio Code Debugging (KU1.4)

Why Debugging?

Debugging allows you to pause program execution, inspect variable values at any point, and trace the flow of your code step-by-step. This is essential for finding bugs.

Key Debugging Concepts

Breakpoint

- A marker on a line of code that tells the debugger: "pause execution here"
- Set by clicking in the gutter (left margin) next to the line number — a red dot appears
- When the program reaches that line, it pauses and you can inspect variables

Watch

- A panel in VS Code where you type variable names
- Shows live values of those variables as you step through the code
- Example: add `str`, `num`, `valid` to Watch to track them in real time

Step Over (F10)

- Executes the current line and moves to the next line
- Does NOT enter into function calls — treats them as one step
- Use when you trust a function and just want to see its result

Step Into (F11)

- Executes the current line and, if it's a function call, enters the function body
- Use when you want to debug inside a specific function

Continue (F5)

- Resumes execution until the next breakpoint (or end of program)

Debugging Workflow

1. Set a breakpoint on the first suspicious line
2. Press F5 to start debugging
3. When paused: check variable values in the **Variables** panel or **Watch** panel
4. Press F10 to step over, F11 to step into
5. Add variable names to **Watch** panel to track them
6. Press F5 again to continue to the next breakpoint

4. Structures (SE3.3)

What is a Struct?

A struct groups together related variables of different types under one name. Think of it as a custom data type — a "record".

```
struct dog {
    char chipId[15];    // 15-char array (NOT a pointer)
    char name[30];      // 30-char array
    char breed[30];     // 30-char array
};
```

Important: Use `char name[30]` (array), NOT `char *name` (pointer), for struct fields you'll write to binary files. Arrays embed the data directly in the struct; pointers store an address, and binary file reads won't reconstruct the pointed-to data.

Declaring and Initialising

```
// Method 1 – declare and initialise at once
struct dog d1 = {"123456789012345", "Buddy", "Labrador"};

// Method 2 – declare then set fields individually
struct dog d2;
scanf("%14s", d2.chipId);
strcpy(d2.name, "Rex");

// Method 3 – via pointer
struct dog *p = &d2;
printf("%s\n", p->name);    // use -> with pointers
printf("%s\n", (*p).name); // equivalent, but -> is cleaner
```

typedef — Cleaner Struct Names

```
typedef struct {
    char chipId[15];
    char name[30];
    char breed[30];
} Dog;

Dog d; // no need for "struct" keyword
```

5. Collections of Structures (SE3.3)

Method 1 — Array of Structs (Static)

```
#define MAX 3
struct dog dogList[MAX]; // fixed size, on the stack

// Fill up
for (int i = 0; i < MAX; i++) {
    dogList[i] = fillRecord();
}

// Display
for (int i = 0; i < MAX; i++) {
    displayRecord(dogList[i]);
}
```

Advantage: Simple, no malloc needed.

Disadvantage: Fixed size — must know at compile time. Cannot resize.

Method 2 — Dynamic List of Static Structs

Size is decided at runtime using `malloc`. Each element in the list IS the struct itself (embedded).

```
#include <stdlib.h>

int size = 3;
struct dog *dogList = (struct dog*)malloc(size * sizeof(struct dog));
if (!dogList) {
    printf("Memory allocation failed!\n");
    return -1;
}

// Fill up using index
for (int i = 0; i < size; i++) {
    dogList[i] = fillRecord();
}

// Display
for (int i = 0; i < size; i++) {
    displayRecord(dogList[i]);
}

// ALWAYS free when done
free(dogList);
dogList = NULL;
```

Advantage: Size decided at runtime; resizable with `realloc`.

Disadvantage: Must manually `free()` — risk of memory leaks.

Method 3 — Dynamic List of Dynamic Structs (pointer-to-pointer)

Each element in the list is a POINTER to a struct (allocated separately). This allows individual records to be deleted cleanly.

```
int capacity = 10;
struct dog **dList = (struct dog**)malloc(capacity * sizeof(struct dog*));

// Set all pointers to NULL first
for (int i = 0; i < capacity; i++) {
    dList[i] = NULL;
}

// Allocate memory for each record before storing
dList[0] = (struct dog*)malloc(sizeof(struct dog));
*dList[0] = fillRecord(); // dereference to store struct

// Display – check for NULL before accessing
for (int i = 0; i < capacity; i++) {
    if (dList[i] != NULL) {
        displayRecord(*dList[i]); // dereference to pass struct
    }
}

// Delete a specific record (e.g., index 1)
free(dList[1]);
dList[1] = NULL; // set to NULL to avoid dangling pointer

// Free everything
for (int i = 0; i < capacity; i++) {
    if (dList[i] != NULL) {
        free(dList[i]);
        dList[i] = NULL;
    }
}
free(dList);
dList = NULL;
```

Advantage: Individual records can be deleted without affecting others; each record's memory is independent.

Disadvantage: More complex code; must free each record AND the list.

Comparison Table

Feature	Array of Structs	Dynamic List (Static Structs)	Dynamic List (Dynamic Structs)
Size	Fixed at compile time	Set at runtime	Set at runtime
Memory location	Stack	Heap	Heap (list + each record)

Feature	Array of Structs	Dynamic List (Static Structs)	Dynamic List (Dynamic Structs)
Delete individual record	Requires shifting	Requires shifting / leaves gap	Set pointer to NULL + free
Syntax complexity	Simple	Moderate	Complex
Memory management	Automatic	<code>free(list)</code>	<code>free(each record) + free(list)</code>
Pointer type	—	<code>struct dog *</code>	<code>struct dog **</code>

6. Functions (AA4.1 / SE4.3)

Function Anatomy

```
// Return type  Name      Parameters
struct dog fillRecord(void) {
    struct dog d;
    // ... fill d ...
    return d;          // returns a COPY of the struct
}

void displayRecord(struct dog d) { // receives a COPY
    printf("%s\n", d.name);
}
```

Pass by Value vs Pass by Pointer

Pass by value: a copy of the variable is made. Changes inside the function do NOT affect the original.

```
void tryChange(int x) {
    x = 99; // only changes the local copy
}
int num = 5;
tryChange(num);
// num is still 5
```

Pass by pointer: the address of the variable is passed. Changes via dereference DO affect the original.

```
void change(int *x) {
    *x = 99; // dereferences pointer to change original
}
int num = 5;
change(&num); // pass the address using &
// num is now 99
```


isNumeric Function (full implementation)

```
#include <ctype.h>
#include <string.h>

_Bool isNumeric(char *str) {
    if (!str || strlen(str) == 0) return 0;
    for (int i = 0; str[i] != '\0'; i++) {
        if (!isdigit(str[i])) return 0; // found a non-digit
    }
    return 1; // all characters were digits
}
```

toNumeric Function — Data In/Out Explanation

```
_Bool toNumeric(char *s, int *number) {
    if (!s || !isNumeric(s) || strlen(s) > 10) return 0;
    *number = atoi(s); // write result through pointer
    return 1;
}

// In main:
int num = 0;
char str[] = "123";
_Bool valid = toNumeric(str, &num);
```

How data is passed in and out:

- `char *s` — the string is passed BY POINTER (address of first character). The function can read the string but cannot meaningfully change where `s` points from main's perspective.
- `int *number` — a POINTER to `num` in main is passed. When the function does `*number = atoi(s)`, it dereferences the pointer and writes the result directly into main's `num` variable. This is how the converted integer is returned "out" of the function — via a pointer parameter, not a return value.
- The return value `_Bool` is used to signal success/failure (1 or 0). Two pieces of data come "out": the converted number (via pointer) and whether it worked (via return value).

7. Binary Files (SE3.3 / SE4.3)

What are Binary Files?

Binary files store data in raw bytes — exactly how it is stored in memory. This is the most efficient way to save struct data to disk because `fwrite` writes the entire struct in one call.

Writing Structs to a Binary File

```
#include <stdio.h>
#include <stdlib.h>

// Assume struct dog is defined and dogList is filled

FILE *fp = fopen("dogData.bin", "wb"); // "wb" = write binary
if (!fp) {
    printf("Error opening file!\n");
    return -1;
}

for (int i = 0; i < count; i++) {
    if (fwrite(&dogList[i], sizeof(dogList[i]), 1, fp) != 1) {
        printf("Error writing record %d\n", i);
        fclose(fp);
        return -1;
    }
}

fclose(fp);
printf("Data saved to dogData.bin\n");
```

`fwrite(pointer_to_data, size_of_one_element, number_of_elements, file_pointer)`

Reading Structs from a Binary File (with dynamic list that grows)

```
FILE *fp = fopen("dogData.bin", "rb"); // "rb" = read binary
if (!fp) {
    printf("Error opening file!\n");
    return -1;
}

int capacity = 10;
struct dog *dogList = (struct dog*)malloc(capacity * sizeof(struct dog));

struct dog temp;
int count = 0;

while (fread(&temp, sizeof(temp), 1, fp) == 1) {
    if (count >= capacity) {
        capacity *= 2;
        dogList = (struct dog*)realloc(dogList, capacity * sizeof(struct dog));
        if (!dogList) { printf("Realloc failed\n"); return -1; }
    }
    dogList[count] = temp;
    count++;
}

fclose(fp);
```

`fread(pointer_to_buffer, size_of_one_element, number_to_read, file_pointer)`
Returns the number of elements successfully read. Returns less than 1 at end of file.

File Open Modes Summary

Mode	Meaning
"wb"	Write binary (creates file, overwrites if exists)
"rb"	Read binary
"ab"	Append binary
"w"	Write text (overwrites)
"r"	Read text
"a"	Append text

8. Text Files (SE4.3)

Writing to a Text File

```
FILE *fp = fopen("log.txt", "a");    // "a" = append, "w" = overwrite
if (!fp) {
    printf("Cannot open file\n");
    return -1;
}

fprintf(fp, "Dog saved: %s\n", dogName);    // formatted write
fputs("Another line\n", fp);                // simple string write

fclose(fp);
```

`fprintf` — like `printf` but writes to a file. Supports format specifiers.

`fputs` — writes a string without formatting.

Reading from a Text File

```
FILE *fp = fopen("log.txt", "r");
if (!fp) {
    printf("Cannot open file\n");
    return -1;
}

char buffer[500];
while (fgets(buffer, sizeof(buffer), fp) != NULL) {
    printf("%s", buffer);    // fgets includes the newline \n
}

fclose(fp);
```

`fgets(buffer, max_chars, file_pointer)` — reads one line at a time, stops at newline or EOF. Includes the `\n` in the buffer.

9. Pointers (SE3.3)

Pointer Basics

```
int num = 50;
int *p = &num;    // p holds the ADDRESS of num

printf("%d\n", num);    // value: 50
printf("%p\n", p);      // address stored in p
printf("%d\n", *p);     // dereference: value at that address = 50

*p = 99;            // change num through pointer
printf("%d\n", num);  // now 99
```

Pointer Arithmetic

When you increment a pointer, it jumps by the SIZE of the data type it points to.

```
int arr[3] = {10, 20, 30};
int *p = arr;    // points to arr[0]
p++;            // moves 4 bytes forward (size of int) → arr[1]
printf("%d\n", *p);    // 20

char ch = 'A';
char *cp = &ch;
cp++;          // moves 1 byte (size of char)
```

NULL Pointer Check — Always Do This

```
struct dog *ptr = malloc(sizeof(struct dog));
if (ptr == NULL) {
    printf("Allocation failed!\n");
    return -1;
}
```

Wild Pointer vs NULL Pointer

- **Wild pointer** — uninitialized pointer; points to random memory. Dangerous.
- **NULL pointer** — pointer set to `NULL`; safe to check before use.

After `free()`, always set pointer to `NULL` to avoid a **dangling pointer** (pointer to freed memory).

```
free(ptr);
```

```
ptr = NULL;    // prevents accidental use of freed memory
```

10. Dynamic Memory — malloc, realloc, free

```
// malloc - allocate memory
int *list = (int*)malloc(5 * sizeof(int));

// realloc - resize existing allocation
list = (int*)realloc(list, 10 * sizeof(int)); // grow to 10

// free - release memory
free(list);
list = NULL;
```

realloc automatically:

1. Finds a new block of the requested size
2. Copies old data to the new block
3. Frees the old block

Warning: If `realloc` fails, it returns `NULL` and the original block is still valid. Always check the return value.

```
int *temp = (int*)realloc(list, newSize * sizeof(int));
if (!temp) {
    printf("Realloc failed!\n");
    // list is still valid here
    free(list);
    return -1;
}
list = temp;    // only update list if realloc succeeded
```